Claude Certified Architect – Foundations (CCA-F)

Kompakt-Spickzettel für die Prüfung · 5 Domains · 30 Task Statements · 6 Szenarien · Fokus: *Welches Design ist richtig – und warum?*

60 Items · 120 Min · Bestehen ab 720/1000
≈ 2 Min pro Frage · 4 von 6 Szenarien · jede Frage beantworten

Prüfung auf einen Blick

Format	**Nur Single Choice**: 1 richtige Antwort + 3 plausible Distraktoren (meist eine *echte* Technik, falsch angewendet)
Szenarien	Pro Termin werden **4 der 6** Szenarien zufällig gezogen; jedes rahmt eine Gruppe von Fragen → **alle 6 vorbereiten**
Bewertung	Scaled Score 100–1.000, **Pass 720**, Ergebnis Pass/Fail. Unbeantwortet = falsch, **kein Malus** fürs Raten
Rahmen	Proctored, closed book, ohne KI. Anmeldung über Anthropic Academy (Skilljar), Claude Partner Network. Start: März 2026
Admin	Gebühr (berichtet ~$99), Retakes, Gültigkeit, Code (CCA-F / CCAR-F) **vor Buchung auf der Academy prüfen**

Gewichtung der Domains

D1 27 % | D2 18 % | D3 20 % | D4 20 % | D5 15 %

Domain	%	Tasks
D1 Agentic Architecture & Orchestration	**27**	1.1–1.7: Loops, Multi-Agent, Subagent-Kontext, Enforcement/Handoff, SDK-Hooks, Decomposition, Sessions
D2 Tool Design & MCP Integration	18	2.1–2.5: Tool-Interfaces, strukturierte Fehler, Tool-Verteilung/ `tool_choice`, MCP-Server, Built-in Tools
D3 Claude Code Configuration & Workflows	20	3.1–3.6: CLAUDE.md, Commands/Skills, Path Rules, Plan Mode, Iteration, CI/CD
D4 Prompt Engineering & Structured Output	20	4.1–4.6: Kriterien, Few-Shot, Schemas, Validation-Retry, Batch, Multi-Pass-Review
D5 Context Management & Reliability	15	5.1–5.6: Kontext, Eskalation, Fehlerweitergabe, große Codebases, Human Review, Provenance

Die 6 Szenarien – was dort typischerweise geprüft wird

S1 Customer Support Resolution Agent Agent SDK + MCP-Tools `get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`; Ziel 80 %+ First-Contact-Resolution	Agent Loop, **Prerequisite Gate** (Identität vor Refund), Hooks, Tool-Design, Eskalation, Handoff, Case-Facts
S2 Code Generation with Claude Code	CLAUDE.md-Hierarchie, Rules, Skills/Commands, Plan Mode vs. direkt, iterative Verfeinerung
S3 Multi-Agent Research System Coordinator → Websuche, Dokumentanalyse, Synthese, Report mit Zitaten	Hub-and-Spoke, Kontextübergabe, parallele Subagents, Fehlerweitergabe, Provenance
S4 Developer Productivity	Built-in Tools (Read/Write/Edit/Bash/Grep/Glob), Codebase-Exploration, Kontextdegradation, Sessions, MCP
S5 Claude Code for CI/CD	`-p`, JSON-Output, unabhängige Review-Instanz, explizite Kriterien, False Positives, Batch vs. synchron
S6 Structured Data Extraction	Tool-Use-Schemas, `tool_choice`, nullable Felder, Validation-Retry, Few-Shot, Human Review, Kalibrierung

Die 4 Lösungsmuster der Prüfung

1. **Deterministisch schlägt probabilistisch**, wenn eine *Garantie* verlangt ist („muss immer", „nie", Compliance, Geld, Identität) → Hook, Gate, Permission, Schema – **nicht** ein stärkerer Prompt.
2. **Ursache auf der richtigen Ebene beheben:** falsches Tool → *Tool-Beschreibung*; erfundene Werte → *nullable* Feld; Subagent fehlt Info → *explizit übergeben*.
3. **Verhältnismäßig:** Die einfachste Lösung, die das Problem *vollständig* löst, schlägt Klassifikator, Fine-Tuning, zusätzliche Agent-Ebene, größeres Modell/Kontextfenster.
4. **Qualifier lesen:** „most effective *first* step", „most reliable", „lowest cost while meeting SLA" – mehrere Optionen sind *möglich*, nur eine ist *am besten*.

Die 10 Entscheidungsregeln

Wenn das Szenario sagt …	… ist die Antwort meist
„must always", „never", Compliance, Geld, Identität	**Deterministische Kontrolle**: Hook, Prerequisite Gate, Permission, Schema
Modell wählt das falsche Tool	**Tool-Beschreibungen** verbessern (Zweck, Inputs, Abgrenzung); überlappende Tools umbenennen/splitten
Subagent fehlt Info, die der Coordinator hatte	**Explizit im Delegations-Prompt übergeben** – Subagents erben keinen Kontext
Output muss valides JSON für ein Downstream-System sein	**Tool Use mit JSON Schema** (Structured Outputs), danach Semantik im Code validieren
Modell erfindet Werte, die nicht in der Quelle stehen	Feld **optional/nullable**, „unclear" erlauben
Dieselbe Session hat Code erzeugt und reviewt	**Unabhängige Review-Instanz** ohne das Reasoning des Generators
Nachts / wöchentlich / nicht blockierend, große Mengen	**Message Batches API** (−50 %, ≤ 24 h). Blockierender Pre-Merge-Check → synchron
Kunde verlangt einen Menschen	**Sofort eskalieren** – nicht erst selbst lösen
Subagent-Aufruf schlägt fehl	**Strukturierten Fehlerkontext** an den Coordinator; kein Fake-Erfolg, kein Totalabbruch
Lange Session vergisst Zahlen & Daten	**Persistenter Case-Facts-Block** statt progressiver Zusammenfassungen

D1 Agentic Architecture & Orchestration 27 %

1.1 Agentic Loop – die erwartete Form

```
messages = [{"role": "user", "content": task}]
while True:
    resp = client.messages.create(model=MODEL,
max_tokens=4096,
                                  tools=TOOLS,
messages=messages)
    messages.append({"role": "assistant",
"content": resp.content})
    if resp.stop_reason != "tool_use":   # end_turn
→ fertig
        break
    results = [run_tool(b) for b in resp.content
               if b.type == "tool_use"]
    messages.append({"role": "user", "content":
results})
    # tool_result-Blöcke mit passender tool_use_id
```

`stop_reason`	Bedeutung → Aktion
`tool_use`	Tools gewünscht → **alle** ausführen, Ergebnisse anhängen, erneut aufrufen
`end_turn`	Fertig → Loop verlassen, Antwort zurückgeben
`max_tokens`	Budget mitten in Antwort erschöpft → erhöhen / fortsetzen, **nicht** als fertig werten
`pause_turn`	Server-Tool-Loop pausiert → Antwort zurückschicken, Claude macht weiter
`refusal`	Abgelehnt → behandeln/anzeigen, nicht blind wiederholen

1.1 Anti-Patterns im Agent Loop

Prüfen, ob die Antwort „fertig klingt" · stoppen, sobald ein Textblock kommt · **Iterations-Cap als primäres Stoppsignal** (Cap = Sicherheitsnetz, das flaggt/wirft, kein stilles „done") · Tool-Results (auch Fehler) nicht in die History schreiben · nur einen von mehreren `tool_use`-Blöcken beantworten.

1.2 Multi-Agent-Orchestrierung (Hub-and-Spoke)

- **Coordinator** zerlegt, delegiert, leitet **alle** Kommunikation, aggregiert, entscheidet über weitere Iterationen. Subagents reden **nie direkt** miteinander (Beobachtbarkeit, Konfliktlösung).
- **Isolation:** Jeder Subagent hat eigenen Kontext; sieht nur Delegations-Prompt + eigenen System-Prompt, Tools (ggf. CLAUDE.md).
- **Coverage-Lücke** trotz guter Subagent-Arbeit → Fehler liegt in der **Zerlegung des Coordinators** (zu eng), nicht bei den Subagents.
- Aufwand skaliert mit der Frage: einfache Faktenfrage ≠ volle Pipeline.

1.3 Subagent-Aufruf & Kontextübergabe

- Coordinator braucht das **Task/Agent-Tool in `allowed_tools`** – sonst kann er nicht delegieren („delegiert nie").
- `AgentDefinition(description, prompt, tools)` : `description` steuert, *wann* delegiert wird.
- **Explizit übergeben:** bisherige Findings, Constraints, Quellen-Metadaten, erwartetes Output-Format.
- Rückgabe als **strukturierte Daten mit Metadaten** (URL, Datum, Confidence).
- **Parallel:** mehrere Task-Aufrufe **in einer Antwort** emittieren.

```
options = ClaudeAgentOptions(
  allowed_tools=["Task", "Read", "Grep", "Glob"],
  agents={"web-researcher": AgentDefinition(
    description="Searches the web for one sub-question.",
    prompt="Return JSON: [{claim, source_url, published,
            excerpt}]. Never drop dates.",
    tools=["WebSearch", "WebFetch"])})
```

Falle
„Größeres Kontextfenster / größeres Modell für den Subagent" – irrelevant, die Info wurde nie übergeben. Subagents teilen **keine Konversation** mit dem Coordinator (höchstens das Dateisystem).

1.4 Workflow-Enforcement & Handoff

- **Prerequisite Gate** im Code: `process_refund` blockiert, bis `get_customer` eine verifizierte ID geliefert hat. Gate gibt **strukturierten Fehler** zurück (was fehlt) – kein stilles Blocken.
- Mehrteilige Anfragen (Retoure + Adressänderung + Rechnungsfrage) in Teilanliegen zerlegen, jedes abarbeiten.
- **Handoff an Menschen:** knappe strukturierte Zusammenfassung (Kunden-ID, Bestell-IDs, Beträge, Bisheriges, Ursache, empfohlene Aktion) – **kein Transkript-Dump**.

Falle
„System-Prompt verschärfen / Few-Shot / GROSSBUCHSTABEN" bei Compliance-Fehlern → bleibt probabilistisch (Fehlerrate > 0). Gate oder Hook wählen.

1.5 Agent SDK Hooks

Bedarf	Mechanismus
Refunds > $500 nur durch Menschen	**PreToolUse**: blockieren + umleiten (Grund nennen!)
Tools liefern Unix-Zeit, ISO, „MM/DD", Statuscodes 3 vs. „shipped"	**PostToolUse**: normalisieren, bevor das Modell es sieht
Third-Party-MCP-Server, nicht änderbar	Hook um das Tool herum
Stil / Vorlieben / Vorgehen	Prompt oder CLAUDE.md (reicht, wenn keine 100 % nötig)

```
async def refund_policy(input_data, tool_use_id, context):
    if input_data["tool_input"]["amount"] > 500:
        return {"hookSpecificOutput": {
          "hookEventName": "PreToolUse",
          "permissionDecision": "deny",
          "permissionDecisionReason":
            "Over $500: call escalate_to_human instead."}}
    return {}
options = ClaudeAgentOptions(hooks={"PreToolUse": [

HookMatcher(matcher="mcp__billing__process_refund",
            hooks=[refund_policy])]})
```

Der **Grund** geht an Claude zurück → Agent eskaliert statt zu wiederholen. Claude Code: Hooks = Shell-Commands in `settings.json`, **Exit 2 blockiert**; Agent SDK: Callbacks in `ClaudeAgentOptions(hooks=…)`.

1.6 Task Decomposition

Fester Prompt Chain	vorhersehbare, immer gleiche Schritte (Checkliste) → billiger, schneller, zuverlässiger als ein Agent
Dynamisch / adaptiv	offene Untersuchung → Agent plant und re-plant je nach Fund
Großes Review	Pro-Datei-Pässe (lokal) + **ein Cross-File-Integrationspass** – statt ein Mega-Pass über 40 Dateien

1.7 Sessions: Resume, Fork, Neustart

Operation	Wie	Wann
Resume	`claude --resume <id|name>`, `--continue`; SDK `resume=`	gleiche Arbeit, Kontext noch gültig
Fork	SDK `resume=…, fork_session=True`; CLI `--fork-session`	**divergente Ansätze** vom gleichen Stand vergleichen, ohne Kontamination
Neu + Summary	neue Session mit strukturierter Zusammenfassung	Tool-Results **veraltet** / Kontext zugemüllt

Wenige bekannte Dateien geändert → resumen und **genau sagen, was sich geändert hat** („re-read billing.py"). Nie das alte Transkript in eine neue Session kippen. Sessions benennen (`/rename`).

D2 Tool Design & MCP Integration 18 %

2.1 Tool-Interfaces

- Die **Beschreibung ist das primäre Auswahlsignal**: Zweck, Input-Formate, Beispiel-Queries, Edge Cases, **wann dieses statt des ähnlichen Tools**.
- Generische Tools splitten: `analyze_document` → `extract_data_points`, `summarize_content`, `verify_claim_against_source`. Überlappende umbenennen.

Falle
Als erster Fix „Routing-Klassifikator bauen" (overengineered), „Tools zusammenlegen" (versteckt Mehrdeutigkeit in Parametern) oder Few-Shot als Hauptfix (Tokens pro Call, skaliert nicht). **Erst Beschreibungen.**

2.2 Strukturierte Fehler für MCP-Tools

```
{ "isError": true,
  "content": [{ "type": "text", "text":
    "
{\"errorCategory\":\"transient\",\"isRetryable\":true,
      \"message\":\"Order service timed out after 5s\",
      \"attempted\":\"lookup_order(ORD-1182)\"}" }]
}
```

Kategorie	Beispiel	Agent soll
Transient	Timeout, 503	Retry mit Backoff
Validation	falsches ID-Format	Input korrigieren, retry
Business	Refund über Policy	erklären/eskalieren, **kein Retry**
Permission	nicht autorisiert	eskalieren, **kein Retry**

Leer ≠ fehlgeschlagen
„0 passende Bestellungen" = erfolgreiches Ergebnis. „DB nicht erreichbar" = Fehler. **Nie `[]` bei einem Fehler zurückgeben** (stiller Fehler als Erfolg getarnt = fast immer die schlechteste Option). Generisches „Operation failed" verhindert passende Recovery. Lokal recovern, bevor man weiterreicht.

2.3 Tool-Verteilung & `tool_choice`

- Zu viele Tools verschlechtern die Auswahl: **4–5 fokussierte** pro Agent schlagen 18. Tools nach Rolle scopen; für häufigen Cross-Role-Bedarf ein **schmales** Zusatz-Tool.

`{"type": "auto"}`	darf auch nur Text antworten
`{"type": "any"}`	muss **irgendein** Tool aufrufen → strukturiert, wenn Dokumenttyp unbekannt (Claude wählt Schema)
`{"type": "tool","name":"x"}`	muss **x** aufrufen → bekannter, fester Schritt
`{"type": "none"}`	kein Tool

Falle
`auto`, obwohl „immer strukturierte Daten" verlangt ist. Forced Tool, obwohl das richtige Tool vom Input abhängt (→ `any`).

2.4 MCP-Server-Integration

Scope	Ort	Geteilt?
Project	`.mcp.json` im Repo-Root	**Ja** – versioniert, fürs Team
User	`~/.claude.json`	Nein – persönlich / experimentell
Secrets	`"env": {"TOKEN": "${GITHUB_TOKEN}"}`	zur Laufzeit expandiert – **nie Tokens committen**

- Tools **aller** konfigurierten Server sind gleichzeitig verfügbar (Namensschema `mcp__server__tool`).
- **MCP Resources** für Content-Kataloge (Doku, Schemas): Sichtbarkeit ohne explorative Tool-Calls.
- Für Standard-Integrationen (GitHub, Jira …) **bestehende Server** nutzen statt eigene bauen.

2.5 Built-in Tools

Grep	Datei-**Inhalte** durchsuchen: Aufrufer einer Funktion, Fehlerstrings, Imports
Glob	Dateien nach **Pfadmuster**: `**/*.test.tsx` (nie für Aufrufer!)
Read / Write	Datei laden / ganze Datei schreiben
Edit	gezielte Änderung über **eindeutigen** Anker-String; nicht eindeutig → mehr Kontext oder **Read + Write** (nicht denselben Edit wiederholen)
Bash	Befehle, Tests, Builds

Inkrementell erkunden (Grep → gezielt Read) statt die ganze Codebase vorab zu lesen.

D3 Claude Code Configuration & Workflows 20 %

3.1 CLAUDE.md-Hierarchie

Was	Wo	Geladen
Persönliche Präferenzen	`~/.claude/CLAUDE.md`	immer, **nur für dich** (nicht geteilt)
Team-Konventionen	`CLAUDE.md` / `.claude/CLAUDE.md`	immer, für alle (versioniert)
Paket-/Bereichsregeln	`<dir>/CLAUDE.md`	beim Arbeiten in dem Verzeichnis
Dateityp-Regeln über Verzeichnisse	`.claude/rules/*.md` mit `paths:`	nur wenn passende Dateien berührt werden
Modulare Includes	`@path/to/file.md` in CLAUDE.md	mit der einbindenden Datei
Team-Workflow on demand	`.claude/skills/<n>/SKILL.md`, `.claude/commands/*.md`	bei Aufruf / Relevanz
Persönlicher Workflow	`~/.claude/skills/`, `~/.claude/commands/`	nur für dich

Fallen
Team-Regeln in `~/.claude/CLAUDE.md` (nur dein Rechner) · eine Riesen-CLAUDE.md im Monorepo (splitten: `@import`, Verzeichnis-Dateien, Rules) · „Anweisung wird ignoriert" → zuerst mit `/memory` prüfen, ob die Datei überhaupt geladen ist, nicht umformulieren.

3.2 Slash Commands & Skills

```
--- .claude/skills/review-api/SKILL.md
name: review-api
description: Review an API route against our REST
  conventions. Use when asked to review endpoints.
argument-hint: "[path to route file]"
allowed-tools: Read, Grep, Glob
context: fork
---
Review $ARGUMENTS against docs/api-conventions.md.
Report only violations, with line numbers.
```

- `context: fork` → läuft in isoliertem Subagent-Kontext; verbose Skills fluten sonst den Hauptkontext.
- `allowed-tools` → „nur auditieren, nicht ändern" deterministisch machen.
- `argument-hint` / `$ARGUMENTS` → Parameter.
- **Skill** = On-Demand-Workflow · **CLAUDE.md** = immer aktive Konventionen. Team-Command in `.claude/commands/` (Repo), nicht `~/.claude/`.

3.3 Path-specific Rules

```
--- .claude/rules/testing.md
paths:
  - "**/*.test.ts"
  - "**/*.test.tsx"
---
Use Vitest and Testing Library. One behaviour per
test.
```

Wählen, wenn Konventionen **über viele Verzeichnisse verteilt** gelten (z. B. alle Testdateien). Verzeichnis-CLAUDE.md würde Duplikate erfordern; Root-CLAUDE.md lädt Irrelevantes in jede Session; ein Skill wird nicht automatisch angewendet.

3.4 Plan Mode vs. direkte Ausführung

Plan Mode	groß, mehrere Dateien, architektonisch, mehrdeutig, riskant (z. B. Migration, sweeping Refactor) – Alternativen abwägen, bevor Code entsteht
Direkt	klein, klar, eine Datei (Bugfix mit klarer Ursache, Validierung ergänzen)
Explore-Subagent	verbose Discovery isolieren – nur die Zusammenfassung landet im Hauptkontext

Kombinierbar: erst planen, dann ausführen. Plan Mode für jede Kleinigkeit = Overhead.

3.5 Iterative Verfeinerung

- **Konkrete Input/Output-Beispiele** statt längerer Prosa bei mehrdeutigen Transformationen.
- **Test-driven:** fehlschlagende Tests / Stacktrace **einfügen** statt beschreiben.
- **Interview-Pattern:** in unbekannter Domäne erst Claude Fragen stellen lassen.
- Zusammenhängende (interagierende) Probleme **gebündelt** lösen, unabhängige **nacheinander**.

3.6 CI/CD-Integration

```
claude -p "Review the diff against CLAUDE.md.
Report only
  bugs and security issues." --output-format json \
  --json-schema "$(cat review-schema.json)" >
review.json
```

- `-p` / `--print` = nicht-interaktiv; ohne hängt der Job und wartet auf Eingabe.
- `--output-format json` + `--json-schema` → maschinenlesbar (Inline-PR-Kommentare) – kein Regex auf Freitext.
- Review-Kriterien & Testkonventionen in **CLAUDE.md** → CI bekommt denselben Kontext wie Entwickler.
- Review durch **separate Instanz**, nicht die generierende Session.
- **Bisherige Findings mitgeben** → Re-Runs melden nur neue/ungelöste Punkte (keine Duplikat-Kommentare).
- Blockierender Pre-Merge-Check → **synchron**, nie Batch API.

D4 Prompt Engineering & Structured Output 20 %

4.1 Explizite Kriterien & Präzision

- Konkrete **kategoriale Kriterien** statt „sei konservativ / nur wichtige Probleme": was melden, was **überspringen** (z. B. Stil, Naming, TODOs).
- **Severity-Definitionen mit Beispielen**.
- Kategorien mit hoher False-Positive-Rate **vorübergehend abschalten**, um Vertrauen in die präzisen wiederherzustellen; dann nachschärfen.

Falle
„Nur Findings mit Confidence > 0,8" als Hauptfix – selbstberichtete Confidence ist unkalibriert. Erst Kriterien.

4.2 Few-Shot Prompting

- **2–4 gezielte** Beispiele für **mehrdeutige** Grenzfälle, die das **Reasoning** zeigen (nicht nur das Label).
- Formate variieren (Rechnungslayouts) → weniger Halluzination bei neuen Layouts; demonstriert Output-Format.
- 10 offensichtliche Beispiele lehren die Grenze nicht. Few-Shot ersetzt **kein Schema** (macht JSON wahrscheinlich, nicht garantiert).

4.3 Structured Output via Tool Use & JSON Schema

```
tools=[{"name": "record_invoice",
  "input_schema": {"type": "object",
    "properties": {
      "invoice_no": {"type": "string"},
      "due_date": {"type": ["string", "null"]},
      "category": {"enum": ["goods","services",
                            "other","unclear"]},
      "category_detail": {"type":
["string","null"]},
      "stated_total": {"type": "number"},
      "calculated_total": {"type": "number"},
      "conflict_detected": {"type": "boolean"}},
    "required": ["invoice_no", "stated_total"]}}],
tool_choice={"type": "tool", "name":
"record_invoice"}
```

- Schema eliminiert **Syntaxfehler, nicht semantische** → Summen, Datumsplausibilität im Code prüfen.
- **Required + nicht-nullable** für evtl. fehlende Infos → **Fabrikation**. Nullable/optional verwenden.
- Enums mit `"other"` + Detail-String und `"unclear"`; Normalisierungsregeln (Datumsformat, Währung) festlegen.
- Nur „antworte in JSON" im Prompt reicht für Downstream-Integration nicht.

4.4 Validation, Retry & Feedback

- Retry mit **Dokument + fehlerhaftem Output + konkretem Validierungsfehler** (ohne Fehler → derselbe Fehler).
- Retries helfen **nicht**, wenn die Info **nicht im Dokument steht** → Retries deckeln, an Human Review routen.
- **Self-Check-Felder:** `calculated_total` vs. `stated_total`, `conflict_detected`. Abweichung **flaggen**, nicht stillschweigend überschreiben.
- `detected_pattern`-Felder, um abgelehnte Findings systematisch zu analysieren.

4.5 Message Batches API

Kosten	**50 %** des Standardpreises
Laufzeit	bis **24 h**, **keine Latenz-SLA**
Zuordnung	`custom_id` pro Request – Ergebnisse in **beliebiger Reihenfolge**
Fehler	**Nur fehlgeschlagene `custom_id`s** neu einreichen (Ursache vorher fixen, z. B. zu große Dokumente splitten)
Ungeeignet	blockierende Workflows (Pre-Merge), Multi-Turn-Tool-Loops innerhalb eines Requests
Vorgehen	Prompt erst an einer **Stichprobe** verfeinern, dann der große Lauf

4.6 Multi-Instance & Multi-Pass Review

- Selbst-Review in derselben Session = **Selbstbestätigung** (Reasoning des Generators im Kontext) → **unabhängige Instanz**.
- Pro Datei lokal + Cross-File-Integrationspass statt ein Mega-Pass (ungleichmäßige Tiefe).
- Confidence-Selbstauskunft nur zum **Routen** von Reviews – Schwellen erst auf gelabelten Daten validieren, nie unkalibriert auto-approven.

D5 Context Management & Reliability

5.1 Kritischen Kontext erhalten

- Progressive Summaries verlieren **Zahlen, Daten, Zusagen** → **persistenter Case-Facts-Block** (Beträge, Daten, Bestell-IDs, Commitments) wörtlich außerhalb der zusammengefassten History.
- **Lost in the Middle:** Kernbefunde an den **Anfang**, Section Headers in langen Inputs.
- **Tool-Output trimmen** auf benötigte Felder, bevor er sich ansammelt.

> **Falle**
> „Modell mit größerem Kontextfenster" – verschiebt das Problem nur; Summaries verlieren weiter Zahlen, die Mitte wird weiter vernachlässigt.

5.2 Eskalation & Mehrdeutigkeit

Eskalieren bei	Nicht (allein) bei
Expliziter Bitte um Menschen (sofort!), Policy-Lücke/Ausnahme, kein Fortschritt	Sentiment/Frust-Score (misst Stimmung, nicht Bedarf), selbstberichteter Confidence

Mehrdeutiger Treffer (zwei „J. Smith") → **nach Identifikator fragen** (E-Mail, Bestellnr.), nie den wahrscheinlichsten raten.

5.3 Fehlerweitergabe in Multi-Agent-Systemen

- Subagent liefert **strukturierten Fehlerkontext**: Fehlertyp, versuchte Query, Teilergebnisse, Alternativen.
- **Nie** als leeren Erfolg unterdrücken (falsche Schlüsse ohne Signal); **nie** alles wegen eines Fehlers abbrechen (verwirft valide Arbeit); keine rohen Stacktraces.
- Coordinator: andere Quelle/Retry versuchen, sonst **Coverage-Lücke im Report annotieren**.

5.4 Kontext in großen Codebases

- Degradation in langen Sessions erkennen (widersprüchliche/vergessene Details).
- **Scratchpad-Dateien** mit Findings; verbose Exploration an **Subagents** delegieren.
- Vor Phasenwechsel zusammenfassen; `/compact` **bewusst an Phasengrenzen** mit Anweisung, was erhalten bleibt.
- **Crash-Recovery-Manifeste** (Fortschritt, erledigte Dateien) für lange Läufe.

5.5 Human Review & Confidence-Kalibrierung

- **Aggregierte Accuracy versteckt Schwächen** (97 % gesamt, aber 70 % bei handschriftlichen Formularen) → pro **Dokumenttyp und Feld** messen, bevor Review reduziert wird.
- **Stratifizierte** Stichproben (seltene, fehleranfällige Segmente) statt einfacher Zufallsstichprobe.
- Feld-Confidence-Schwellen auf **gelabeltem Validierungsset** kalibrieren.

5.6 Provenance & Unsicherheit

- **Claim → Quelle** (URL/Dokument, Auszug, Datum) muss Zusammenfassungen überleben.
- Widersprüchliche Werte: **beide behalten, mit Quelle annotieren** – nicht einen auswählen; unterschiedliche Daten/Methoden erklären.
- Veröffentlichungs- und Erhebungsdatum mitführen; Inhaltstypen passend darstellen (Tabelle für Zahlen, Prosa für Erklärungen).

Verwechslungsgefahr

Hook / Gate ↔ Prompt	Garantie ↔ Guidance
PreToolUse ↔ PostToolUse	blockieren/umleiten ↔ normalisieren
Resume ↔ Fork ↔ Neu	weiter ↔ Alternativen ↔ veraltete Daten
Grep ↔ Glob	Inhalt ↔ Pfad
`auto` ↔ `any` ↔ forced	Text erlaubt ↔ irgendein Tool ↔ genau dieses
`.mcp.json` ↔ `~/.claude.json`	Team ↔ persönlich
Rules ↔ Dir-CLAUDE.md ↔ Skill	Dateityp quer ↔ ein Bereich ↔ on demand
Batch ↔ synchron	−50 %, ≤ 24 h ↔ blockierend
Leeres Ergebnis ↔ Fehler	Erfolg mit 0 Treffern ↔ `isError`
Business/Permission ↔ Transient	kein Retry ↔ Retry mit Backoff

Nicht im Fokus (Out of Scope)

Fine-Tuning/Training (RLHF, Constitutional AI), Embeddings & Vektor-DB-Interna, API-Auth & Billing-Admin, Hosting von MCP-Servern, Computer Use/Vision-Details, Streaming-Implementierung, Cloud-Provider-Konfiguration. Prompt Caching / Token Counting: Existenz kennen reicht.

Prüfungstaktik

- **Zeit:** 2 Min/Item → nach 60 Min ≈ Frage 30. Schwere Items flaggen, weiter, am Ende zurück.
- Szenario-Kontext lesen: welche Tools, welche Constraints (Geld? Compliance? Latenz? Team vs. persönlich?).
- Eliminieren: **nur Prompt** bei Garantie-Anforderung · **überdimensioniert** (Klassifikator, Fine-Tuning, größeres Modell/Kontext) · **stiller Fehler** · **am Root Cause vorbei**.
- Keine Frage leer lassen. **Bereit** bei stabil **≥ 80 %** in vollen Mocks über verschiedene Szenario-Ziehungen; offizielle Practice Exam auf der Academy machen.

Last-Minute-Check pro Szenario

S1 Support	Identität per **Gate** vor `process_refund` · Betragslimit per **PreToolUse** · Mensch verlangt → sofort eskalieren · zwei Treffer → ID erfragen · Case-Facts-Block · Handoff = strukturierte Summary · Business-Fehler nicht retryen
S2 Code-Gen	Team → Projekt-CLAUDE.md / `.claude/` · persönlich → `~/.claude/` · Testkonventionen quer → `.claude/rules/` + `paths` · verboser Skill → `context: fork` · ignoriert → `/memory` · großer Umbau → Plan Mode
S3 Research	Lücke → Zerlegung des Coordinators · Kontext explizit übergeben · Task-Tool erlaubt? · parallel = mehrere Calls in einer Antwort · Fehler strukturiert + Lücke annotieren · Claim→Quelle, Konflikte beide behalten
S4 Dev-Prod.	Grep = Inhalt, Glob = Pfad · Edit braucht eindeutigen Anker · inkrementell erkunden · Explore-Subagent · Scratchpad + `/compact` an Phasengrenzen · Fork für Alternativen, Neustart bei veralteten Results · `.mcp.json` + `${ENV}`
S5 CI/CD	`-p` · `--output-format json` + `--json-schema` · separate Review-Instanz · alte Findings mitgeben · explizite Kriterien, laute Kategorien abschalten · Pre-Merge synchron, Nightly → Batch
S6 Extraction	Tool Use + Schema · `any` bei unbekanntem Typ, forced bei festem Schritt · nullable statt Fabrikation · `other` / `unclear` · Retry mit konkretem Fehler, gedeckelt · stated vs. calculated total flaggen · Accuracy pro Typ/Feld, stratifiziert prüfen · Batch: `custom_id`, nur Fehlgeschlagene neu

> **Merksatz**
> Garantie → Code (Hook/Gate/Schema). Auswahl → Beschreibung. Wissen → explizit übergeben. Fehler → strukturiert, nie still. Kosten ohne Eile → Batch. Prüfen → unabhängige Instanz.